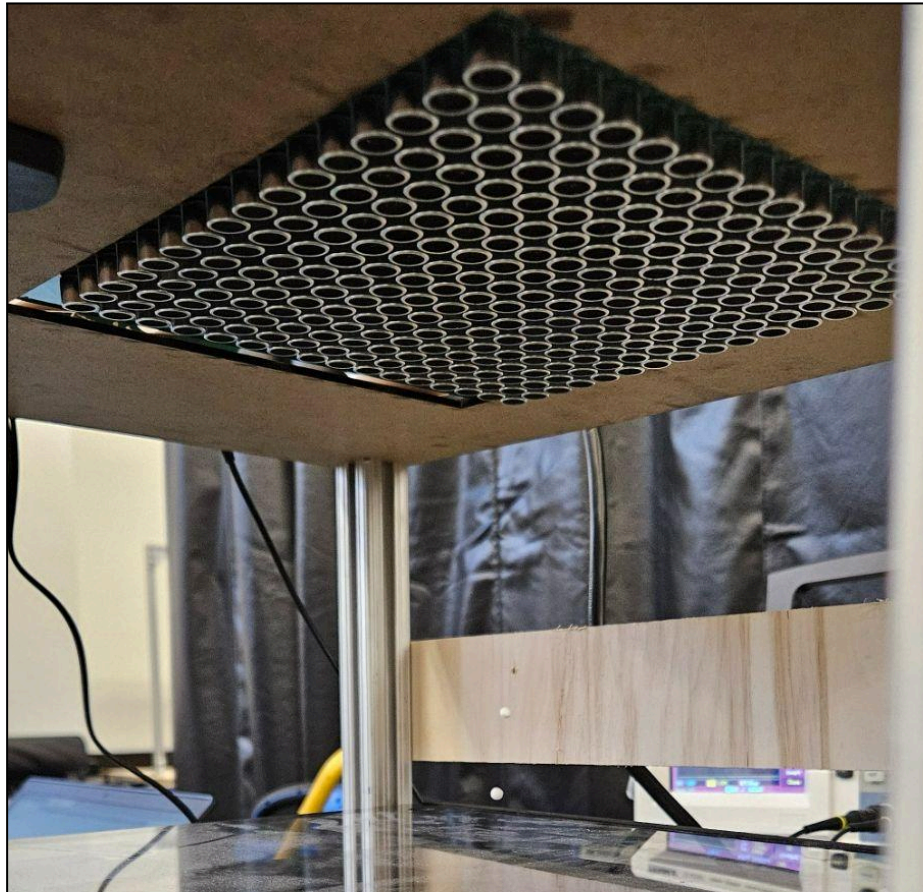


Characterization of a Phased Array for Ultrasonic Levitation



Paige Hall
Zavary Koehn
Casey Krombein
Howard Li

Project Sponsor:
Engineering Physics Project Lab

Project # 2551
ENPH 479
Engineering Physics Project Lab
The University of British Columbia
April 12th, 2026

Executive Summary

This project aims to develop a system for ultrasonic levitation of styrofoam beads, with the end goal of achieving bead movement speeds sufficient for achieving a hologram effect, via persistence of vision. Levitation is achieved by a 16x16 array of small transducers, which are each driven at 40 kHz. By varying the relative phases of the transducers, the standing wave pattern produced can be modified to form a low pressure region in between two high pressure regions, in which a bead can levitate. To this end, we make use of a numerical solver to generate the required phases and two FPGAs to generate the signals to drive the transducers, among other system components.

At the start of this year, we had a working system. However, it had some issues and eventually became non-functional, which prompted us to spend much of the year doing a hardware redesign. The other main priority for this year was developing a comprehensive test plan for moving forward with the project. Additionally, we implemented various other improvements to make the system more robust and easier to test.

The hardware redesign involved splitting the electrical system into two separate boards to increase modularity, as well as upgrading the communications system to increase the theoretical maximum update rate of the system. We ran into significant manufacturing issues with the new hardware, but we were ultimately able to fully verify its functionality.

Our test plan aims to progress through the key parameters of the system in a logical order. First, we test for system repeatability and spatial resolution, as these inform all subsequent tests. Then, we test for the spatial dependence of trap strength, as this is needed to properly interpret all future results. Finally, we test for maximum acceleration and velocity, as these are the key metrics for attaining fast bead movement.

On the software side, we implemented new position generation code featuring a GUI to make testing easier, and we added functionality to pre-generate phases and save them, to improve runtime latency. We also upgraded the mechanical setup to improve stability and reliability, and wrote new video processing code to more easily extract movement data from tests. The goal of all these changes was to allow for easier testing and iteration in the future.

Next steps for the project involve performing more in-depth testing, exploring more complicated control methods, and optimizing the update speed of the system, as well as a number of other potential system improvements which are outlined in the report proper.

Table of Contents

Introduction.....	1
Discussion.....	1
Background.....	1
System Characterization.....	1
Test Plans.....	3
Repeatability Test Plan.....	3
Spatial Resolution Test Plan.....	3
Positional Dependence of Trap Strength Test Plan.....	4
Maximum Acceleration Test Plan.....	5
Maximum Velocity Test Plan.....	6
GUI for Generating System Characterization Tests.....	7
Hardware Redesign.....	9
Updated Phase Solver.....	10
Pressure Field Simulation.....	11
Mechanical Upgrades.....	12
System Level Diagram.....	12
Conclusions.....	13
Recommendations.....	13
Hardware.....	13
Software + Physics.....	14
Deliverables.....	14
References.....	15
Appendix A - Results from Pressure Field Simulation Tests.....	16
Appendix B - Video Processing Code.....	17

Table of Figures

Figure 1: Example of expected result from a repeatability test.....	3
Figure 2: Example of a potential result from a spatial resolution test.....	4
Figure 3: Example of expected results from a positional dependence of trap strength test. The x's represent where the bead fell out of the trap. In reality, the x's would likely be closer to the edge of the array.....	5
Figure 4: Example of expected results from a maximum acceleration test. The x's represent where the bead fell out of the trap.....	6
Figure 5: Example of expected results from a maximum velocity test. The x's represent where the bead fell out of the trap and the orange lines show the tangent at that point. The tangent corresponds to the maximum velocity.....	7
Figure 6: GUI for generating system characterization tests.....	8
Figure 7: Top-down view of FPGA and FTDI chip PCB (annotated).....	9
Figure 8: (left) Bottom side of transducer PCB, depicting the 256 transducers arranged in a 16x16 grid. (right) Top side of transducer PCB, depicting the 128 gate driver ICs required to drive the transducers.....	10
Figure 9: Output from pressure field simulation (x-z cross-section at $y = 0.08$ m), depicting a trap centered near the middle of the levitation region.....	11
Figure 10: Mechanical/physical system setup (annotated).....	12
Figure 11: System Level Diagram.....	13
Table 1: Effect of varying reflection coefficient R on simulated pressure distribution.....	16
Figure 12: (top) Raw frame data, (middle) frame in HSV, (bottom) frame after gaussian binary threshold.....	17

Introduction

The aim of this project is to develop a phased array system capable of high-speed ultrasonic levitation of a styrofoam bead, for applications in hologram generation.

A significant challenge with working on this system is the inability to see or interact with the fields produced by the phased array, and thus, careful planning is required to evaluate system capabilities and to improve system design. The goal of this year's work was to design test plans to characterize the system's performance, identifying performance-limiting factors and informing future system upgrades, with the aim of achieving holograms via the persistence of vision effect.

Additionally, the board we inherited from the previous team became non-functional and we were unable to fully repair it, so we redesigned the hardware, making it more modular and robust, and made improvements to both the mechanical system and software, which are outlined in the remainder of this report.

Discussion

Background

This report assumes that the reader is familiar with the report from the previous year, which describes the fundamental physics of the project and the associated system in detail [1].

As is discussed in the previous report, we want to be able to rapidly trace out arbitrary paths with a levitated bead fast enough to achieve persistence of vision and produce holograms. To achieve this, we need to be able to move the bead with high speeds and accelerations. Simply moving a trap at these high rates does not guarantee the bead will follow because this system does not have closed loop control. Thus, we need to ensure the system is designed to produce traps that are sufficiently strong to overcome any inertial, drag, or environmental forces that seek to eject the bead from the trap at the high speeds and accelerations we desire to use. Furthermore, knowing the maximum speed and acceleration we can move a bead at will set the scope of what holograms the system is capable of producing.

System Characterization

The two key figures of merit we are interested in are maximum acceleration and maximum velocity. These quantities will be spatially dependent within the levitation region due to the finite number of transducers. Regions near the edges of the levitation region are not supported by as many surrounding transducers as the inner regions are, leading to inhomogeneous trap strength throughout the levitation region. Additionally, each transducer has a slightly different amplitude which leads to further field strength inhomogeneities.

Measuring these quantities is challenging since we cannot disturb the field by placing sensors in the levitation region. Thus, we cannot easily visualize or measure the true pressure field distribution in the levitation region during operation. Instead we need to measure the system's key performance metrics through observation alone. The key figures of merit are:

- 1) Maximum acceleration
- 2) Maximum velocity
- 3) Spatial dependence of trap strength

To perform this observation, we use a 240fps iPhone camera; refer to [Video Processing Code](#) for details on the data processing and [Mechanical Upgrades](#) for the setup.

How do we even know there *is* a maximum speed and acceleration? We have observed the bead falling out of the trap, or appearing to jump around before being ejected, during previous experimentation with the system. Given that there is only this one observed failure mode for multiple root causes, we need to ensure we design tests that isolate each fundamental/underlying physical principle leading to failure. Some causes are fundamental physical limits, such as inertia and drag. Others are environmental causes or system artifacts, such as air flow, asymmetry of the bead, or placement in a ghost trap. The tests we design should yield interpretable results that tell us concrete statements about the system's capabilities so we can understand what is limiting system performance without any ambiguity as to the cause of what we are seeing.

The two basic physical principles that influence the maximum speed and acceleration of a levitating bead are inertia and air resistance. When the trap moves a bead, the restoring force from the walls of the trap is what keeps the bead inside the trap. If the trap accelerates so fast that its restoring force is unable to overcome the bead's own inertia, the bead will fall out. Since inertial force is $F = ma$ and is thus related to acceleration, this places a limitation on the maximum acceleration the trap can move with.

If the trap moves so fast that its restoring force can no longer overcome air resistance (drag force), then the bead will fall out. This places a limitation on the maximum velocity the trap can move at because air resistance is dependent on velocity, not acceleration.

Both the acceleration and velocity limitations depend on trap location within the levitation region because they both depend on the trap's restoring force and as discussed above, the restoring force is location dependent due to the finite nature of the system.

Additionally, we want to know the spatial resolution of our system. There is a minimum step size we can perform (smallest movement of a trap) and this limits the resolution of our trap control. This is because we only have 8-bit resolution on our phases, so we can't vary trap location continuously. Understanding this limitation will be important when designing tests to measure the maximum acceleration and velocity of the system, as well as in the future when performing rasterization and charting bead paths. However, it is possible that this limitation is small enough that it is not relevant.

Test Plans

The following test plans are presented in the order they should be conducted. The order was carefully chosen, as each test requires information from prior tests.

Repeatability Test Plan

Before performing any actual testing, it is important for us to understand how robust and reliable our system is. This requires testing how reliably the system can reproduce a given result when environmental conditions are changed. Some relevant environmental conditions would be:

- Repeating the same tests consecutively
- Power cycling the system in between tests
- Taking the bead out and putting it back in
- Different air flows in the room
- Different positioning of the system (i.e. slightly tilted table vs flat)

This will inform us of the natural variation in performance we can expect without changing any operational parameters.

Test plan:

- 1) Run a preset test path
- 2) Vary one condition from the list above
- 3) Rerun the same preset test
- 4) Repeat a few times and quantify the variability
- 5) Repeat for different changing conditions and document which conditions affected the results

For a preset path of [i.e., move forward x cm, stop for t seconds. Move backwards x cm, stop for t seconds.], we expect an output that resembles Figure 1.

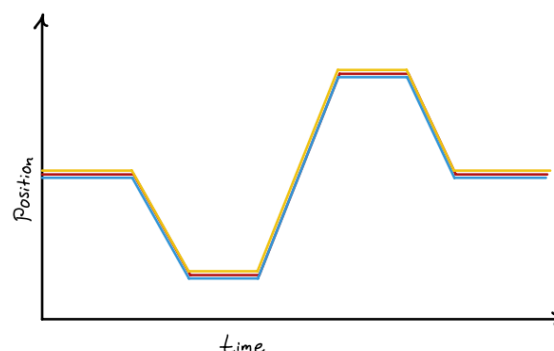


Figure 1: Example of expected result from a repeatability test.

Spatial Resolution Test Plan

Now that we know how reliable the system is, we need to know our minimum step size so that we can ensure we do not violate it in future testing.

Test plan:

- 1) Start the trap in a specific location
- 2) Very gradually increment the step size sent to the solver (starting from the same position each time) starting with a small value that does not cause the trap to move
- 3) When the trap actually moves, stop and measure that step size
- 4) Repeat a few times per location in different locations across the levitation region

We want to start with step sizes much less than the trap size. To determine a good starting step size, we can use our pressure field simulation (see [Pressure Field Simulation](#)) to determine an approximate order of magnitude for our trap size, and then choose a step size accordingly. A satisfactory precision for the minimum step size is likely much smaller than the trap size. If our spatial resolution is on a visible scale, we might expect results as in [Figure 2](#).

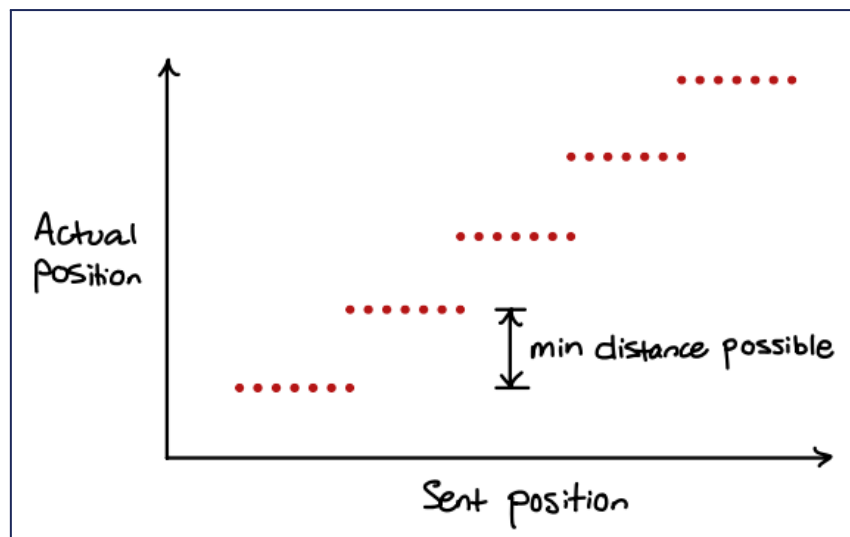


Figure 2: Example of a potential result from a spatial resolution test.

However if our spatial resolution is high quality, we may just see a linear trend.

Positional Dependence of Trap Strength Test Plan

We want to get a measure of the positional dependence of trap strength before proceeding with other tests so we can understand how to interpret results. For this, we need a way to quantify the trap strength. When a bead is perturbed in a trap, it oscillates back and forth. By measuring this oscillation frequency, we can determine the resonant frequency of the trap which is proportional to the restoring force, the acoustic radiation force, which is a measure of trap strength [1].

Test plan:

- 1) Move the trap a small amount (δx)
- 2) Let the particle settle, measure the position vs time, and compute the resonant frequency

- 3) Extract the first order trap strength
- 4) Repeat in different locations

We can determine a good δx using the results of the spatial resolution testing and our knowledge of the trap size. We want to perturb the bead by moving the trap far enough to displace the bead from its equilibrium position by a measurable amount but not too far so as to remove the bead from the trap. We expect results to look similar to [Figure 3](#), not necessarily in shape, but in trend. Experimentally, we noted that the system was able to levitate a bead right up to the edge of the board, in line with the edge row of transducers.

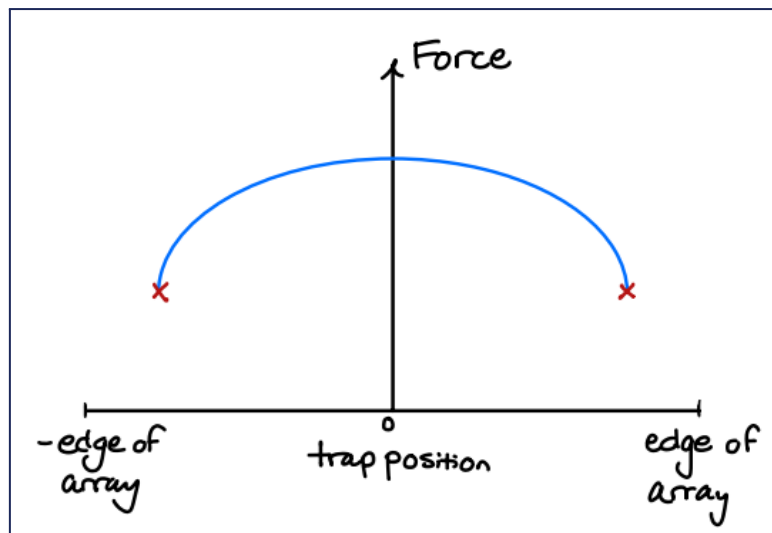


Figure 3: Example of expected results from a positional dependence of trap strength test. The x's represent where the bead fell out of the trap. In reality, the x's would likely be closer to the edge of the array.

Maximum Acceleration Test Plan

Now that we know how repeatable the system is, the smallest step size we can safely take, and the approximate positional dependence of trap strength, we can move on to measuring the key figures of merit.

To measure the maximum acceleration, we will accelerate the trap at different rates until the bead falls out. We should ideally do this over a short distance so that the peak velocities achieved are well below what would likely be the maximum velocity limit. Last year, through observation, we determined an approximate maximum velocity in the center region of the board of $\sim 50\text{cm/s}$ [1]. This maximum velocity is not terribly accurate, but provides a sense of the order of magnitude on which we should choose our distance scales. By ensuring we are operating well below the velocity limit, we can ensure that the failure mode is caused by the inertial limit.

To choose a starting acceleration, from the positional dependence test we know the positional dependence of the trap's resonant frequency:

$$F_{inertial} = F_{restoring} \rightarrow ma = kx$$

Max acceleration occurs when the bead leaves the trap. If we take x as the approximate trap size and approximate the bead mass, using the k from the positional dependence test ($k = m\omega^2$), we can approximate the expected maximum acceleration:

$$a_{max} \sim \frac{kx}{m} = \omega^2 x$$

Here ω is the resonant frequency measured in the positional dependence test. The starting acceleration chosen should be well below this. Pick an arbitrary acceleration step size. Once the bead falls out, smaller acceleration step sizes can be used to achieve a more precise result from more focused tests..

Test plan:

- 1) Move the trap at a constant acceleration over a short distance
- 2) Repeat with incrementally larger accelerations until the bead falls out the trap
- 3) Repeat to confirm max acceleration
- 4) Repeat in different regions to determine spatial dependence to confirm theoretical spatial dependence (we'd expect linear relationship with trap strength spatial dependence if not identical)

We expect results to resemble [Figure 4](#).

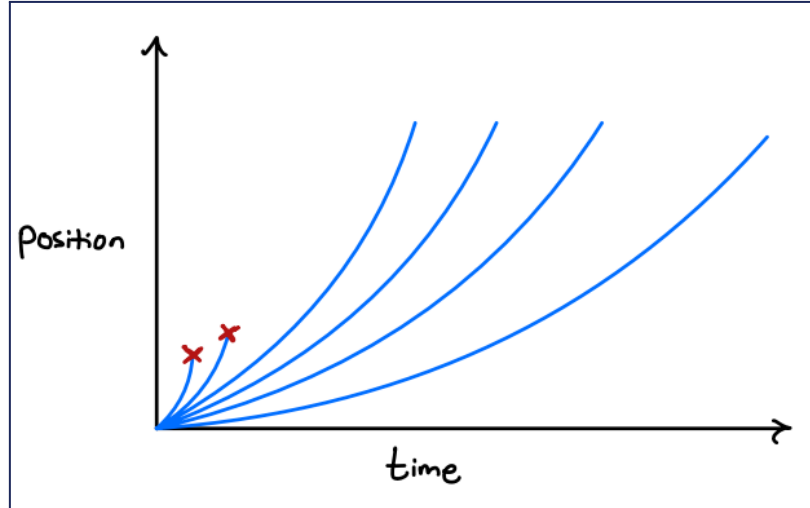


Figure 4: Example of expected results from a maximum acceleration test. The x's represent where the bead fell out of the trap.

Maximum Velocity Test Plan

To measure the drag limit, we need to ensure that the acceleration we use is well below the inertial limit:

$$a_{test} \ll a_{inertial\ max}$$

We want to choose an acceleration that allows the bead to reach an expected escape velocity within a reasonable distance if using linear acceleration. If this distance, given the max acceleration of the system, is not well within the center of the board, the circular acceleration can be used. It is unclear whether the experimentally measured maximum velocity of ~50cm/s was limited by drag or inertia, so we can get an estimate of the expected maximum velocity by looking at the restoring force compared to the drag force. We equate the same restoring force used in our acceleration approximation (kx) with an estimate of the drag force using the standard drag force equation. This gives the following approximation:

$$v \approx \sqrt{\frac{2kx}{\rho C_D A}}$$

Where ρ is the density of error and C_D is the drag coefficient for a sphere.

Test Plan:

- 1) Move the trap at a constant acceleration until the bead falls out (chosen as discussed above)
- 2) Record the velocity at which it fell out (tangent of position-time plot)
- 3) Repeat the above to confirm the max velocity/drag limitation
- 4) Repeat in different regions

We can compare this value with our observed maximum velocity as a sanity check and to confirm if it was valid to use it as an upper limit in the maximum acceleration test. The results of this test should resemble [Figure 5](#).

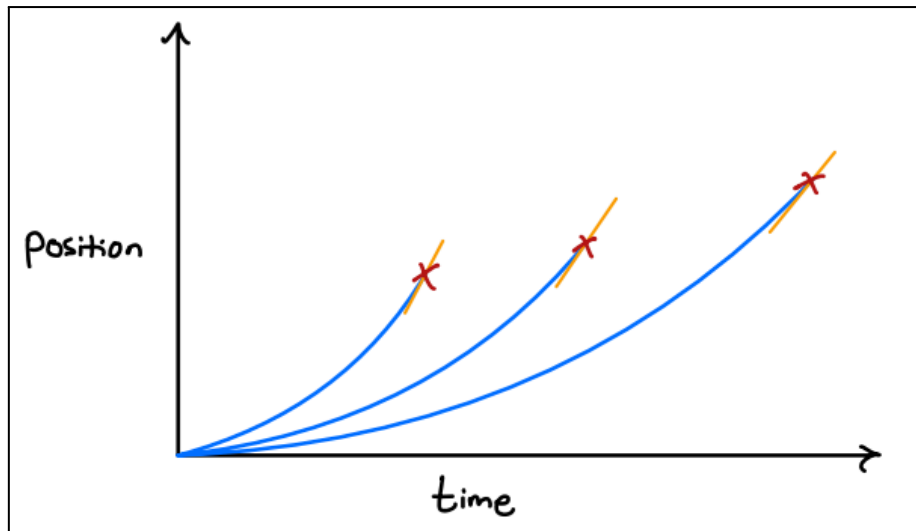


Figure 5: Example of expected results from a maximum velocity test. The x's represent where the bead fell out of the trap and the orange lines show the tangent at that point. The tangent corresponds to the maximum velocity.

GUI for Generating System Characterization Tests

To execute the above-mentioned test plans, we need to control movement of the trap over various distance-, speed-, and time-scales. Thus, we developed a graphical user interface (GUI) with Python and Qt that enables flexible and precise generation of trap movement trajectories, for use in testing/characterizing the system. The GUI serves as an abstraction

layer between the end-user and the underlying phase solver, allowing for complex trap movement without directly modifying low-level, internal code.

The updated Rust phase solver takes in as its input a .csv file, where each row contains four values, (t, x, y, z) , representing the time-evolution of the 3D trap location (x, y, z) at time steps t . Generating such .csv files by hand would be incredibly time-consuming, and system tests will require a diverse set of trap trajectories, which motivates the need for this GUI.

Our design requirements for this GUI were that it:

- Be highly-robust and modifiable in terms of available parameters: from just a handful of parameters, we should be able to quickly generate a large variety of trajectories to implement our test plans.
- Be easy to generate trap trajectories via the frontend GUI, and to add functionality for generating not-yet-implemented trap trajectories via the backend, internal code.
- Provides the end-user with a user-friendly way of visualizing/previewing the movement/trajectory of the trap over time. We hence chose to implement a GUI rather than a purely terminal-based interface.

In satisfying these requirements, we designed the GUI depicted in [Figure 6](#) below.

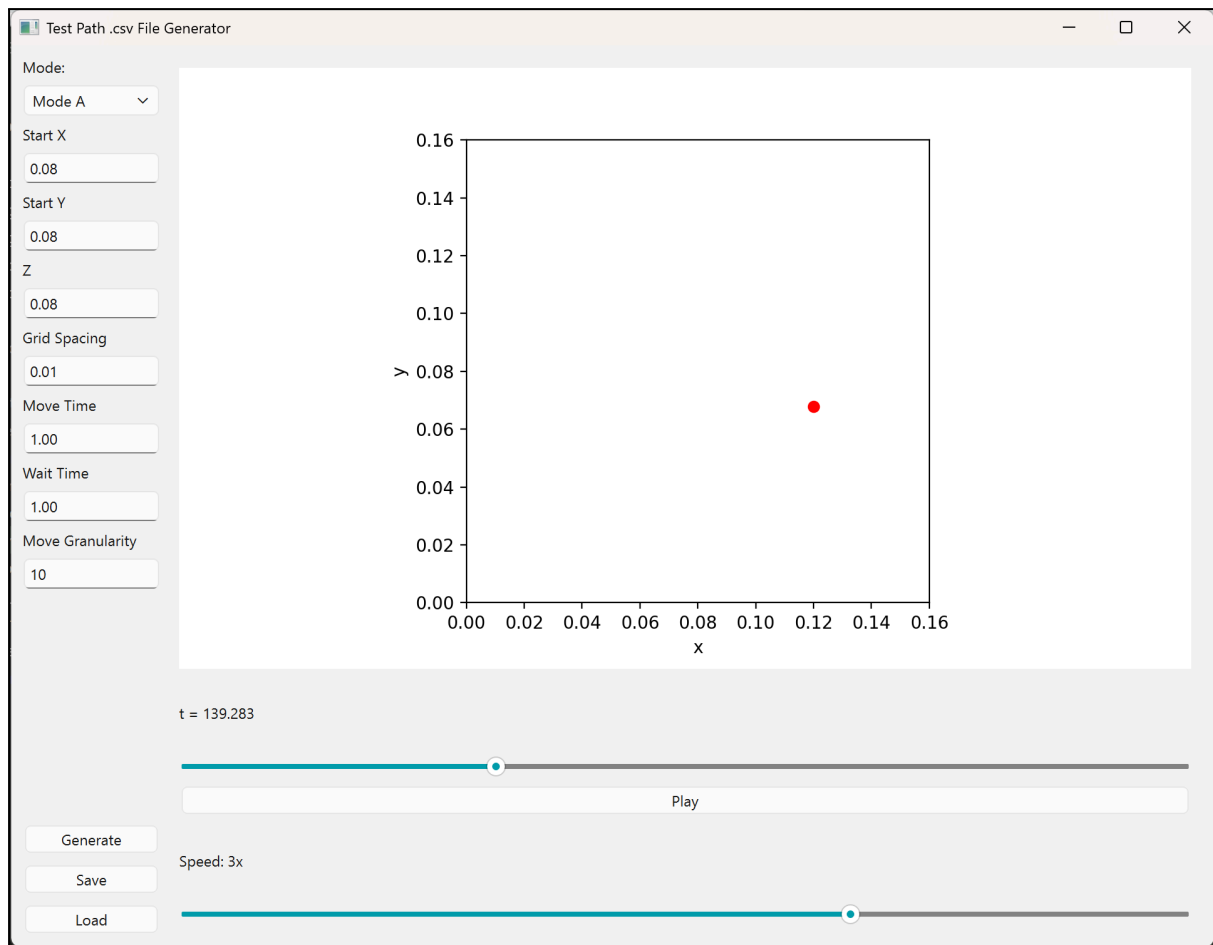


Figure 6: GUI for generating system characterization tests.

On the left, the trajectory mode and parameters are displayed; the available parameters vary depending on the selected mode (each test plan has its own mode). The end-user can easily modify the starting position, speed, and spatial resolution of the trajectory, among other parameters.

The desired trajectory is then generated and can be viewed using the playback feature depicted in the Figure above. The red dot updates with the (x, y) position of the trap, tracing out the generated trap trajectory, which can be exported/imported via .csv file.

In the back-end code, we can:

- Hold the trap still in its current spot for a time span t .
- Move the trap in a straight line between two (x, y) coordinates over a time span t .

Currently, this range of movement functionality is sufficient for executing our test plans, but the code is designed to be easily scalable; movement in circular arcs or along bezier curves, for example, could allow for the implementation of more complex trap trajectories and hence more complex test plans, which may provide further insight into the system's capabilities.

Hardware Redesign

Since the previous hardware stopped working, we decided to design new hardware to improve our reliability and fix a number of outstanding issues. The main difference was splitting the design into two separate PCBs to allow for easier testing and modification.

One PCB contains the FPGAs and the FTDI communications chips, and the other contains the transducers and drivers, as depicted in Figures 7 and 8 below.

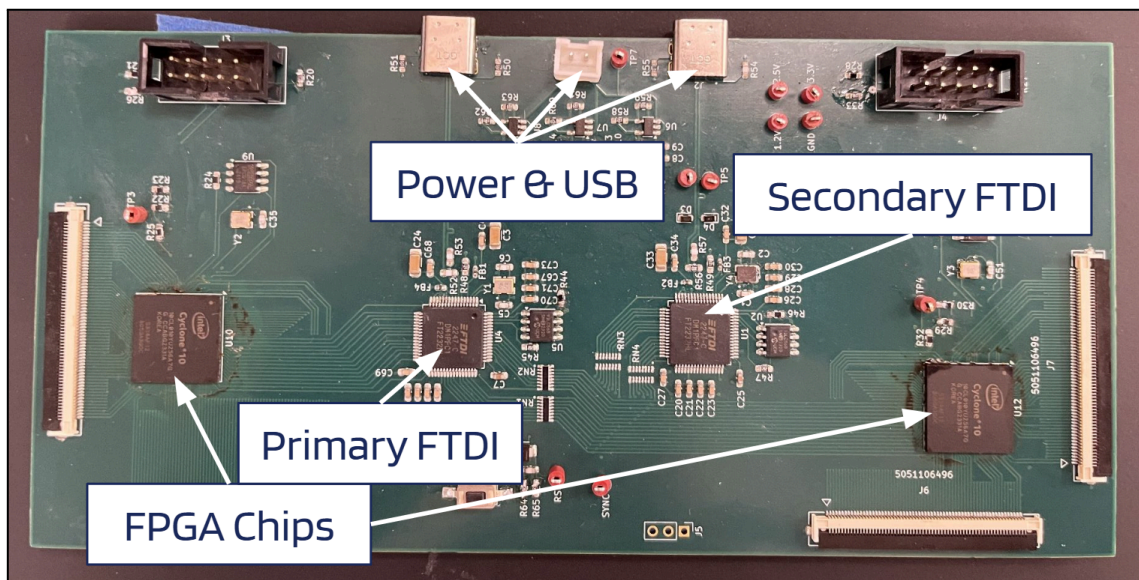


Figure 7: Top-down view of FPGA and FTDI chip PCB (annotated).

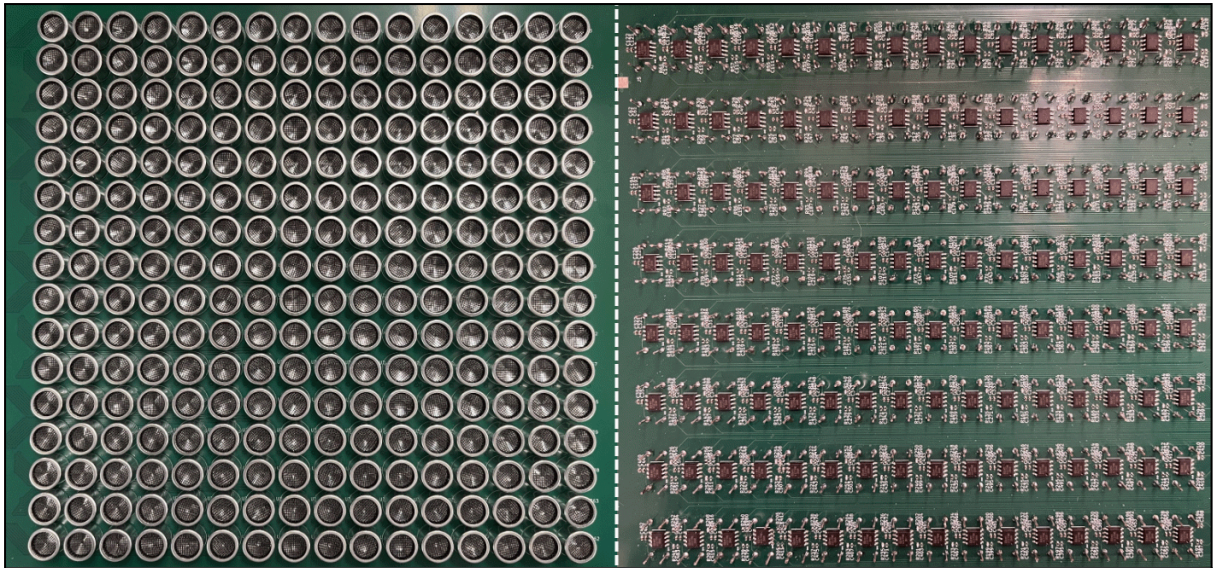


Figure 8: (left) Bottom side of transducer PCB, depicting the 256 transducers arranged in a 16x16 grid. (right) Top side of transducer PCB, depicting the 128 gate driver ICs required to drive the transducers.

The signals for the 256 transducers are sent over four 64 channel ribbon cables.

The next major change made was the use of two FTDI chips instead of one. This gives the option to use the 40MB/s synchronous FIFO mode rather than the 8MB/s asynchronous mode, as the synchronous mode does not support two output channels per chip. This was not fully implemented on the software side, but the increased bandwidth should allow for a faster update speed or greater bandwidth in the future. The board can also be configured to run using only one chip, in which case it is backwards-compatible with the previous software.

All other changes were fixes to minor outstanding issues with the previous board, such as adding missing resistors on the USB connection and fixing incorrect connector footprints, and QOL improvements to make testing and running smoother, such as the addition of more test points on the PCBs.

Updated Phase Solver

The Rust phase solver was updated in order to better align with our goals this year, namely, of executing system characterization tests.

The previous solver took in as its input real-time (x, y, z) trap coordinates, which were provided from Python position input scripts and sent from Python to Rust via the Redis publisher-subscriber framework (more specifically, its Windows equivalent, Memurai). This allowed for direct end-user control of the trap (and hence, the bead) location and was highly useful for demonstration purposes, but required solving for phases in real-time, which introduced a slight latency to the system.

In comparison, the updated solver is modified to take in a .csv file of trap coordinates and times, describing the trajectory of a trap as output by the system characterization test generation GUI. This eliminates the need for real-time solving and for the Redis framework, simplifying program execution. To remove the latency, we pre-compute all the phase offsets required for generating the trap trajectory *before* the initial trap position is sent to hardware. With the phases now stored in memory, we need only use Rust's timing features to publish said phases to the FPGA PCB, at the desired times. Outputs in the Rust terminal indicate that we achieve sub-millisecond level timing precision, which is sufficient for executing system tests. Such precision would be significantly harder to achieve without pre-computation.

The underlying algorithm and calculations remain the same; for this reason, we direct readers who wish to know more about the internals of the solver to previous reports [1] [2].

Pressure Field Simulation

The pressure field simulation takes in as its input the phase offsets of the transducers (as solved by the phase solver) and outputs a 2D cross-section of the pressure distribution in a specifiable sub-section of the levitation region. An example of the output of the simulation is shown in Figure 9 below.

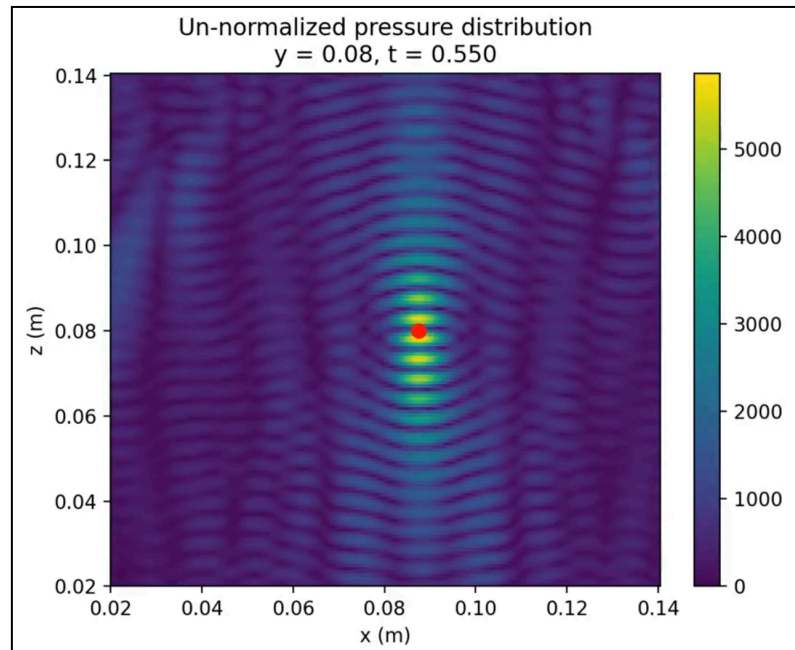


Figure 9: Output from pressure field simulation (x-z cross-section at $y = 0.08$ m), depicting a trap centered near the middle of the levitation region.

This simulation was developed by the previous team [2] (please refer to this citation for more details on the technical implementation), and we did not explore it in much detail last year. This year, we were motivated to further develop the simulation for the purposes of visualization/demoing and for running theoretical tests.

One main improvement was the modification of the simulation to render animations for trap trajectories, as the previous simulation could only output the pressure field at a single time instance. An example animation is provided [here](#).

We also experimented with removing the pressure normalization, in the hopes of obtaining an idea of the scale of the pressures generated by the system. The resulting distribution is in terms of a transducer velocity amplitude U_o (refer to the colorbar at right in [Figure 9](#)), which is an inherent parameter of the transducers used (TCT40-10T/R1). It should be possible to calculate this amplitude from the transducer datasheet, which presents an avenue for further development of the simulation [3].

Lastly, we tested the effect of modifying the reflection coefficient R of the glass plate to values near and significantly below unity; refer to [Appendix A](#) for results.

Mechanical Upgrades

We redesigned the test stand to provide a more level, stable, and easily adjustable test platform. The previous stand was only supported on one end (as how a cantilever is), causing a bend of a few degrees on the far side of the PCB. This was undesirable, as we require the transducers to be as level as possible.

The new stand, pictured in [Figure 10](#), has a 80/20 post in each corner, wooden stabilizing bars to eliminate play/instability, and a height-adjustable scissor lift, upon which sits the reflective glass plate. An iPhone camera mount is attached to the stand to provide an adjustable camera angle for data acquisition.

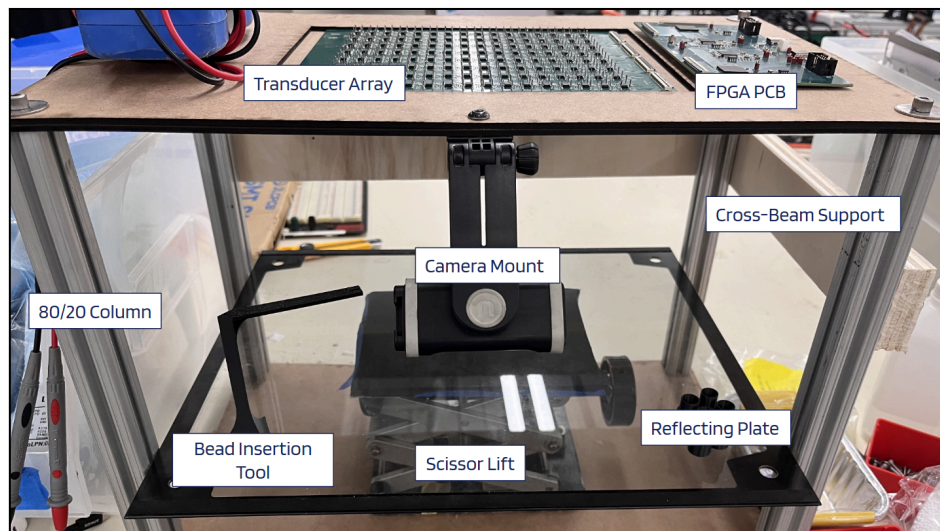


Figure 10: Mechanical/physical system setup (annotated).

System Level Diagram

Now that we have discussed how each individual component of the implementation operates, we return to a high-level overview of the entire system, which is depicted in [Figure 11](#) below.

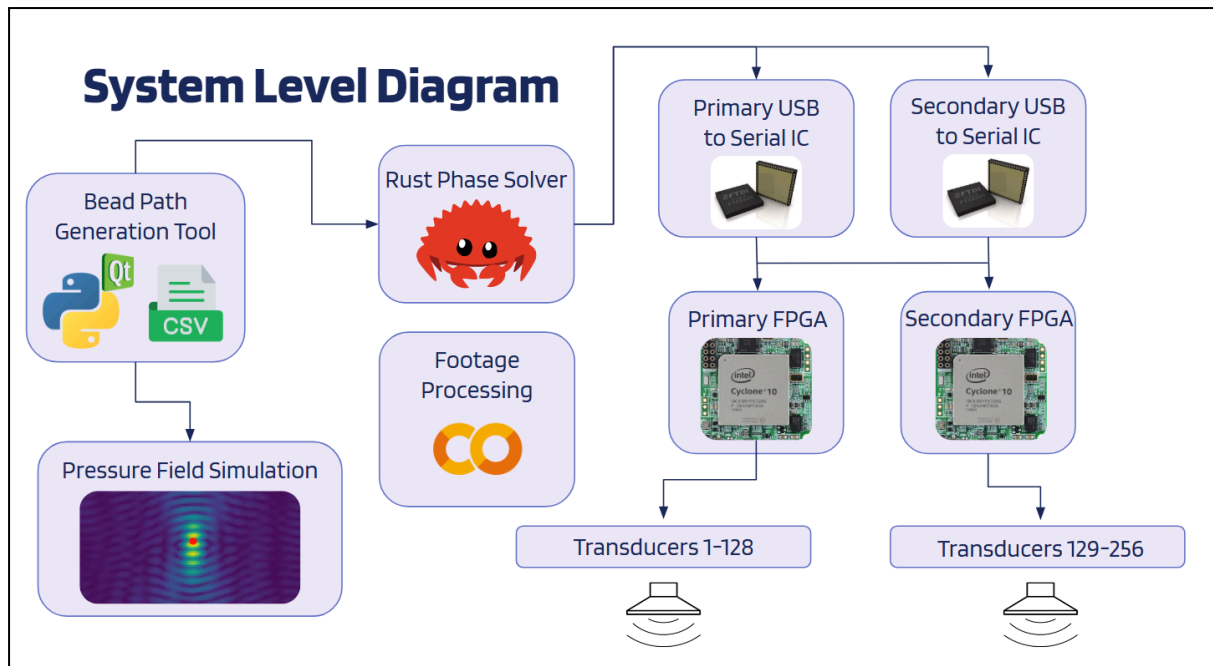


Figure 11: System Level Diagram.

It may be illuminating for the reader to compare this year's diagram to last year's [1].

Conclusions

This year, we aimed to characterize the system by evaluating the key system metrics of maximum speed/acceleration and positional dependence of trap strength. While technical challenges regarding the assembly and operation of both new PCBs prevented us from executing our testing plans within the year, the upgrades made to both the hardware and mechanical stand have significantly improved the reliability of the system. We hope that the documentation provided in this report serves as a guide for future teams to continue the project toward the goal of creating holograms.

Recommendations

Below, we outline our suggestions for this project, based on the content of this report and the observations made/struggles encountered by the team this year:

Hardware

- Create a new transducer board. Either reflow and populate a new version of existing one to fix bad solder joints, or redesign to fix the power connector and add a connector to tie ground to the FPGA board.
 - Would recommend getting non-transducer components populated by the PCB manufacturer, if possible.
- Try adding another transducer board instead of the reflecting plate. This is expected to significantly increase control of particles.

- Implement synchronous FIFO communication with the FPGAs to increase update frequency and perform more detailed testing + optimization of the update rate.

Software + Physics

- Use testing data to design a more sophisticated control scheme for generating trap trajectories. Rather than simply placing traps along the path at a given rate, explore taking into account particle inertia and trap force profile to move the bead more smoothly.
- Investigate using a less simplified model for the transducer pressure profile in the solver to improve accuracy.
- Implement amplitude calibration as well as phase calibration by accounting for transducer amplitude variation in the solver.
 - We observed that there is a transducer-to-transducer variation of up to 20% in terms of transducer output amplitude. Alternatively, additional transducers with less variation in amplitude could be sourced.
- Use simulations and testing to explore what would be required to achieve vertical movement, and implement it, if possible. This may require investigating different solver algorithms, as the trap shape produced by the current one is not conducive to vertical movement (creates a trap that is wide horizontally, but narrow vertically).
- Add functionality to the solver to solve for phases under the assumption that certain transducers are turned off. This would model cases where hardware issues prevent a small fraction of the transducers from powering on.

Deliverables

We submit the following items as project deliverables:

1. This report, which describes both the fundamentals and details of our project and represents the culmination of our work over the past two years.
2. Our main [GitHub](#) repository, containing old files created by the previous team, related to system design, and new files created by us.
3. Separate GitHub repositories for the [system characterization test generation GUI](#), [updated Rust phase solver](#), and [updated pressure field simulation](#).
 - a. The decision to host our new software in separate repositories was made with modularity in mind; each repository runs as its own standalone program.

We also pass down the physical setup, consisting of (but not limited to) the FPGA and transducer PCBs, USB connections/power cords, and the ThinkPad laptop, which houses all our relevant programs and files.

References

- [1] P. Hall, Z. Koehn, C. Krombein, and H. Li, "Ultrasonic Levitation for Holographic Imaging," Apr. 2025.
- [2] A. Chen, Q. Ferbers, K. Lin, and J. Seto, "Pushing the Frontier of Open-Source Ultrasonic Phased Arrays for Multi-Modal Stimuli," Apr. 2024.
- [3] "10PCS/Lot TCT40 10mm 40KHZ 10mm ultrasonic sensor probe 40KHz receiving and transmitting split black metal shell ranging TCT10 - AliExpress 502," AliExpress., 2025. <https://www.aliexpress.com/item/1005008121247973.html> (accessed Apr. 12, 2026).

Appendix A - Results from Pressure Field Simulation Tests

As a sanity check, we simulate pressure distributions for a trap in the center of the levitation region, with reflection coefficient R varying from unity to near-zero. We expect the quality of the trap to degrade as the coefficient lowers, ultimately failing to produce a trap when R is close to zero.

From the pressure field simulation, we obtain the field distributions shown in Table 1 below.

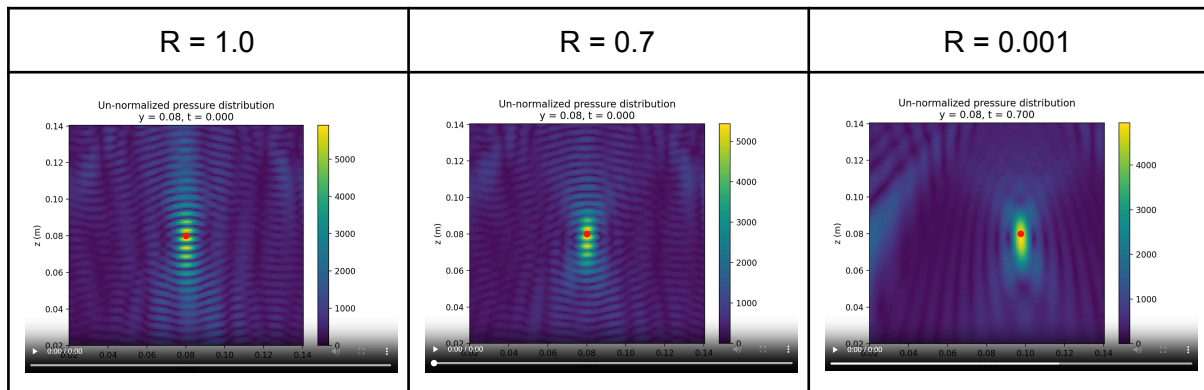


Table 1: Effect of varying reflection coefficient R on simulated pressure distribution.

We see that the cleanest trap is generated at $R = 1.0$; the trap quality degrades at $R = 0.7$ (in both shape and magnitude), and we do not have a trap at all at $R = 0.001$, which is the result that we expect. This implies that the pressure field simulation is producing an accurate depiction of the real pressure field, and that the reflective plate is indeed necessary for the formation of standing waves and pressure nodes/antinodes.

Appendix B - Video Processing Code

To extract figures of merit from test run footage, we developed a video processing Python environment via Google Colab. The code relies on the OpenCV library to first convert the video frame-by-frame to HSV color space, then applies a binary threshold, filtering out low saturation regions. The bead position can then be tracked using the *findContours* function. Multi-object tracking is possible using the current code implementation, but is not necessary for any of the previously mentioned test plans. An example of this processing pipeline is demonstrated in Figure 12 below.

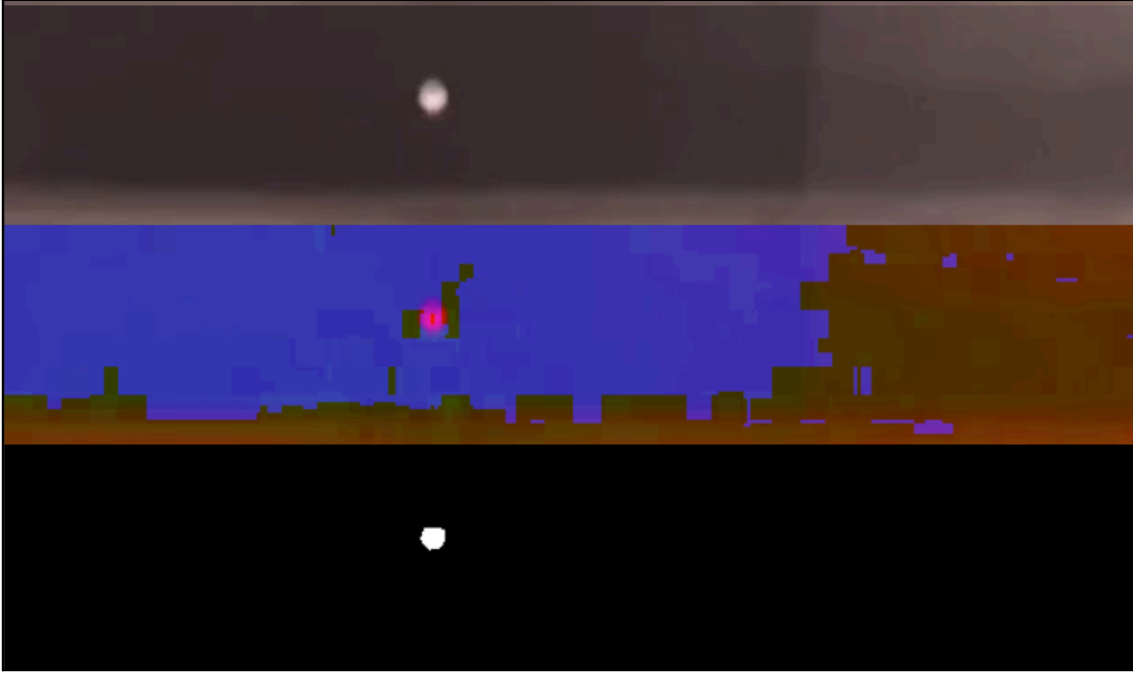


Figure 12: (top) Raw frame data, (middle) frame in HSV, (bottom) frame after gaussian binary threshold

Once the position data is obtained, we can calculate velocity using a standard formula for the derivative:

$$v = \frac{dx}{dt} = \frac{x_{final} - x_{initial}}{t_{final} - t_{initial}},$$

where $x_{initial}$ is the position of the bead in the current frame and x_{final} is the position of the bead, a given number of frames ahead.

As for the acceleration, two different methods are implemented to be able to compare results. The first is found based on the previously calculated velocity data:

$$a = \frac{v_{final} - v_{initial}}{t_{final} - t_{initial}}$$

The second uses the second-order central difference equation:

$$a = \frac{x[i+k] - 2x[i] + x[i-k]}{(\Delta t)^2}$$

Both resultant accelerations are plotted separately. A Gaussian filter to remove noise resulting from discretization when plotting both velocity and acceleration in Colab. Both the smoothed and raw data for position, velocity, and acceleration can then be exported to Microsoft Excel for further processing, as we did last year [1].